

机器学习实战

兵王问题专项练习（SVM）

涵盖内容：

- ✓ UCI krkopt 数据集读取与理解
- ✓ 字符坐标编码与标签二值化
- ✓ 特征标准化 / Z-score 归一化
- ✓ 支持向量机二分类训练
- ✓ 准确率、混淆矩阵、结果可视化

环境：VS Code + Python + MiniConda

请先阅读背景知识与引导性问题，再逐步完成编码

第 1 题 数据集读取与可视化初步

考点：pandas 读取 CSV • UCI krkopt • 样本分布观察

1.1 背景知识

兵王问题是经典的国际象棋残局二分类任务：棋盘上仅剩

- 黑王 (King)
- 白王 (King)
- 白兵 (Pawn)

棋局结果只有两类：

- 和棋 (draw)：黑方可以通过规则保证不输
- 白胜：白方在有限步内可以将死黑王

UCI 提供的 `krkopt.data` 数据集共 28056 条样本，每条样本格式类似：

`a,1,b,3,c,2,draw`

其中前 6 个字段分别对应：

`wx,wy,wx,wy,vx,vy`

黑王 x , 黑王 y , 白王 x , 白王 y , 白兵 x , 白兵 y

最后一个字段是结果标签。

1.2 引导性问题

在写代码前请先回答：

1. 数据集样本总数 28056，其中和棋约 2796 条，白胜约 25260 条。这是类别不平衡问题吗？正负样本比例大概是多少？
2. 6 个坐标特征原本是 `a,b,c,d,e,f,g,h` 与 `1,2,...,8` 的组合，它们是字符串类型。如果直接送入 SVM，能得到数值距离吗？你打算如何把它们映射为整数？
3. 使用 Python 的 `pandas` 读取文件时，如果原始数据没有表头，你需要用哪个参数指定列名？写出列名建议。
4. 用 `value_counts()` 统计标签分布，画出条形图。从分布上看，哪一类占绝对多数？模型即使全猜这个多数类，准确率是多少？
5. 如果没有做任何标准化，不同量纲的特征会影响 SVM 的核函数计算吗？为什么？

1.3 题目要求

实现代码片段，读取 CSV 并打印基本信息：

```
1 import pandas as pd
2
3 # TODO: 填写合适的列名列表
4 col_names = []
5 df = pd.read_csv('krkopt.data', names=col_names)
6
7 print("数据集形状: ", df.shape)
8 print("前5行: ")
9 print(df.head())
10 print("标签分布: ")
11 # TODO: 统计标签分布并打印
```

1.4 提示

下载数据集可以直接从 UCI Machine Learning Repository 搜索 `krkopt`，下载后放在你的 VS Code 工程目录下即可。

第 2 题 坐标特征编码与标签二值化

考点：字符映射替换 • 正则替换 • 二分类标签编码 0/1

2.1 背景知识

原始 6 个坐标特征中：- 横列 `a-h` 可以映射到 1-8 - 纵列 1-8 已经是数字

为了后续送入机器学习模型，必须把所有特征转为整数。同时为了做二分类，把标签列中的 `draw` 映射为 1（正样本：和棋），所有其他结果（`one,six,...` 等白胜）映射为 0（负样本：非和棋）。

2.2 引导性问题

1. 用 `pandas.replace(regex=True,...)` 把列中的字符 `a-h` 替换成 1-8。写出映射规则。为什么正则表达式写 `'^a$'`：1 能精确替换，不会出现意外子串？
2. 替换完成后，所有坐标列仍然是字符串类型。用 `.astype(int)` 转换为整数，打印数据类型，验证是否成功。
3. 标签列替换规则：如果标签等于 `draw` 就返回 1，否则返回 0。这样处理后，二分类任务就定义为“预测该残局是否为和棋”。请用一句话解释：为什么这比预测 16 类步数更简单？

4. 编码完成后，取前 3 条样本手动验证坐标映射是否正确。例如样本 `a,1,b,3,c,2,draw` 应该变成什么数值特征？
5. 编码后，特征矩阵 X 的形状是多少？标签 y 的形状是多少？

2.3 题目要求

补全预处理代码：

```
1 # 字符坐标映射为整数
2 replace_rules = {
3     '^a$': 1, '^b$': 2, '^c$': 3, '^d$': 4,
4     '^e$': 5, '^f$': 6, '^g$': 7, '^h$': 8
5 }
6 coord_cols = ['wx', 'wy', 'wwx', 'wwy', 'vx', 'vy']
7
8 # TODO: 用 df.replace(regex=True) 执行替换
9 # TODO: 将 coord_cols 转换为整数
10
11 # 标签二值化: draw -> 1, 其他 -> 0
12 # TODO: 用 apply + lambda 完成
13 # df['lable'] = ...
14
15 print(df['lable'].value_counts())
```

第 3 题 Z-score 标准化与数据集切分

考点：StandardScaler • 训练集统计量 • 避免数据泄露 • train_test_split

3.1 背景知识

SVM 对特征尺度非常敏感：如果某一特征数值范围很大而另一特征很小，核函数中的欧氏距离会被大尺度特征主导，使分类效果恶化。因此通常采用 Z-score 标准化：

$$x' = \frac{x - \mu}{\sigma}$$

其中 μ 和 σ 是训练集上的均值和标准差。注意：测试集不能反过来修改标准化参数，否则会发生数据泄露。

3.2 引导性问题

1. 为什么只用训练集的均值和标准差去标准化测试集，而不是合并两者之后再计算？这样做犯了什么方法论错误？

2. 用 `train_test_split` 切分数据时,设定 `test_size=0.2` 表示什么?设定 `random_state=42` 的好处是什么?
3. 标准化后,训练集中每个特征理论上均值约为 0, 标准差约为 1。写代码验证这一点。
4. 如果不做标准化,直接在原始坐标 1-8 上训练 SVM, 决策边界会和标准化后的有什么区别?
5. 本任务中正负样本不平衡,你在 `train_test_split` 里设置 `stratify=y` 有什么好处?

3.3 题目要求

补全切分和标准化代码:

```
1 from sklearn.model_selection import train_test_split
2 from sklearn.preprocessing import StandardScaler
3
4 X = df[coord_cols].values
5 y = df['lable'].values
6
7 # 切分 80% 训练, 20% 测试
8 X_train, X_test, y_train, y_test = train_test_split(
9     X, y, test_size=0.2, random_state=42, stratify=y
10 )
11
12 scaler = StandardScaler()
13 # TODO: 在训练集上 fit, 然后分别 transform 训练集和测试集
14
15 print("训练集的均值: ", X_train_scaled.mean(axis=0).round(2))
16 print("训练集的标准差: ", X_train_scaled.std(axis=0).round(2))
```

3.4 验证点

打印出的训练集每个特征均值应该接近 0, 标准差接近 1。

第 4 题 SVM 二分类训练与结果分析

考点: SVC • RBF 核 • 准确率 • 混淆矩阵 • 可视化

4.1 背景知识

支持向量机（SVM）在二分类任务中通过最大化间隔寻找最优分界面。在兵王这张数据集上，常用 RBF 核（高斯核）就能得到很高的准确率。完成训练后，用准确率（Accuracy）、混淆矩阵（Confusion Matrix）、以及分类报告全面评估模型。

4.2 引导性问题

1. SVM 的 C 参数越大，代表对误分类惩罚越重，决策边界倾向于更复杂还是更简单？C 非常大时，训练集过拟合风险上升还是下降？
2. RBF 核参数 `gamma` 控制单个样本的影响范围。`gamma` 过大，决策边界会非常局部化，容易出现什么现象？
3. 混淆矩阵输出四个数：TN, FP, FN, TP。在本问题中，“和棋”作为正类，FP 表示把白胜误判为和棋，FN 表示把和棋误判为白胜。哪一类错误在实际残局判断中可能更危险？
4. 如果不对类别不平衡做任何处理，即使模型全猜“白胜”也能拿到约 90% 准确率。如何通过混淆矩阵发现这种“偷懒”模型？
5. 尝试把标准化后的特征输入模型，再把未标准化的特征输入模型，比较两个准确率。哪一组更高？为什么？

4.3 题目要求

训练 SVM 并输出评估指标：

```
1 from sklearn.svm import SVC
2 from sklearn.metrics import (
3     accuracy_score, confusion_matrix, classification_report
4 )
5
6 clf = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
7 # TODO: 用标准化后的特征训练模型
8 # TODO: 用标准化后的特征进行预测
9
10 print("测试集准确率：", accuracy_score(y_test, y_pred))
11 print("混淆矩阵：")
12 print(confusion_matrix(y_test, y_pred))
13 print("分类报告：")
14 print(classification_report(y_test, y_pred))
```

4.4 预期结果参考

在合理预处理下，你将观察到测试准确率超过 97%，混淆矩阵中非对角元素很小，说明模型可以可靠地预测“兵王残局是否为和棋”。

附：运行建议与常见误区

1. 先在 MiniConda 环境创建新环境。执行：

```
conda create -n chess python=3.10 -y
conda activate chess
pip install pandas numpy scikit-learn matplotlib
```

然后在 VS Code 右下角切换解释器到这个环境，即可运行所有代码。

2. 不要在替换坐标时忘记转整数。如果你发现特征类型还是 object/string，SVM 训练会直接报错。
3. 标准化时切记 fit 只用 `X_train`。很多同学不小心写成 `scaler.fit(X)` 然后再切分，这是典型数据泄露，虽然准确率看起来高，但方法论错误。
4. 不要一开始就把 SVM 参数 `C` 和 `gamma` 调得极端大。建议先用默认值跑通，再做参数对比实验。
5. 可视化标签分布是好习惯。用 `df['lable'].value_counts().plot(kind='bar')` 可以直观提醒自己数据不平衡情况，不要被表面的高准确率迷惑。